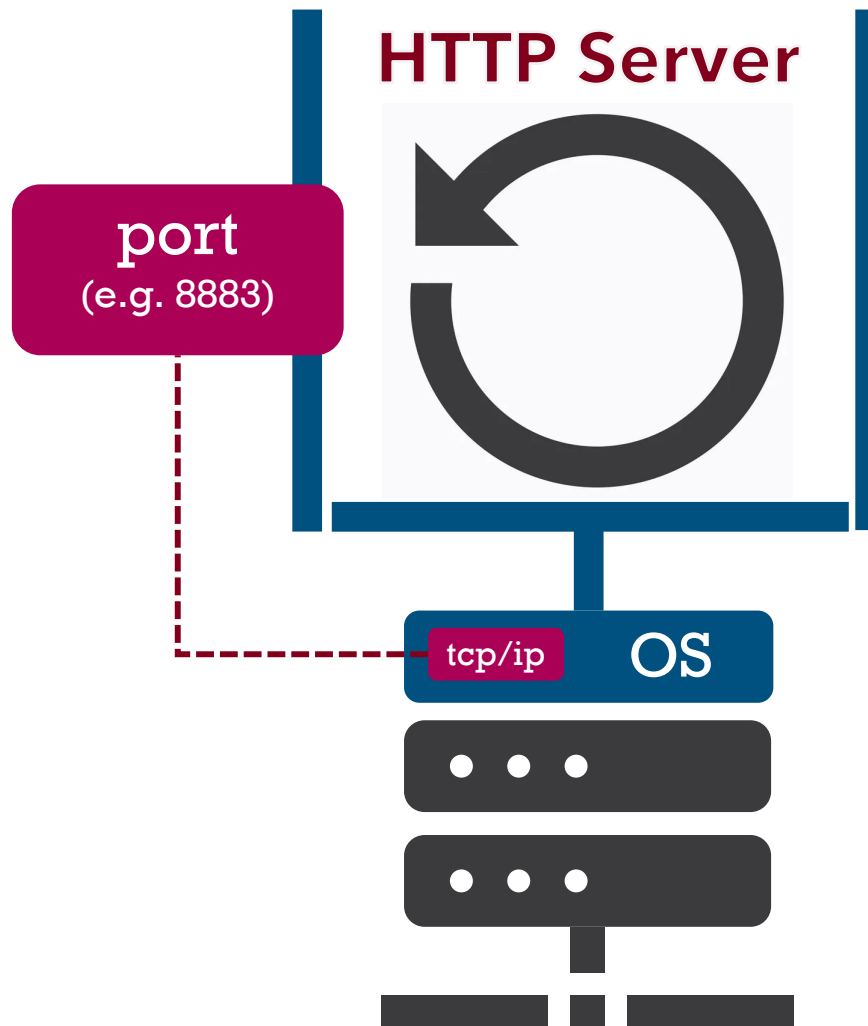




5 – Server HTTP per server-side applications

HTTP Server

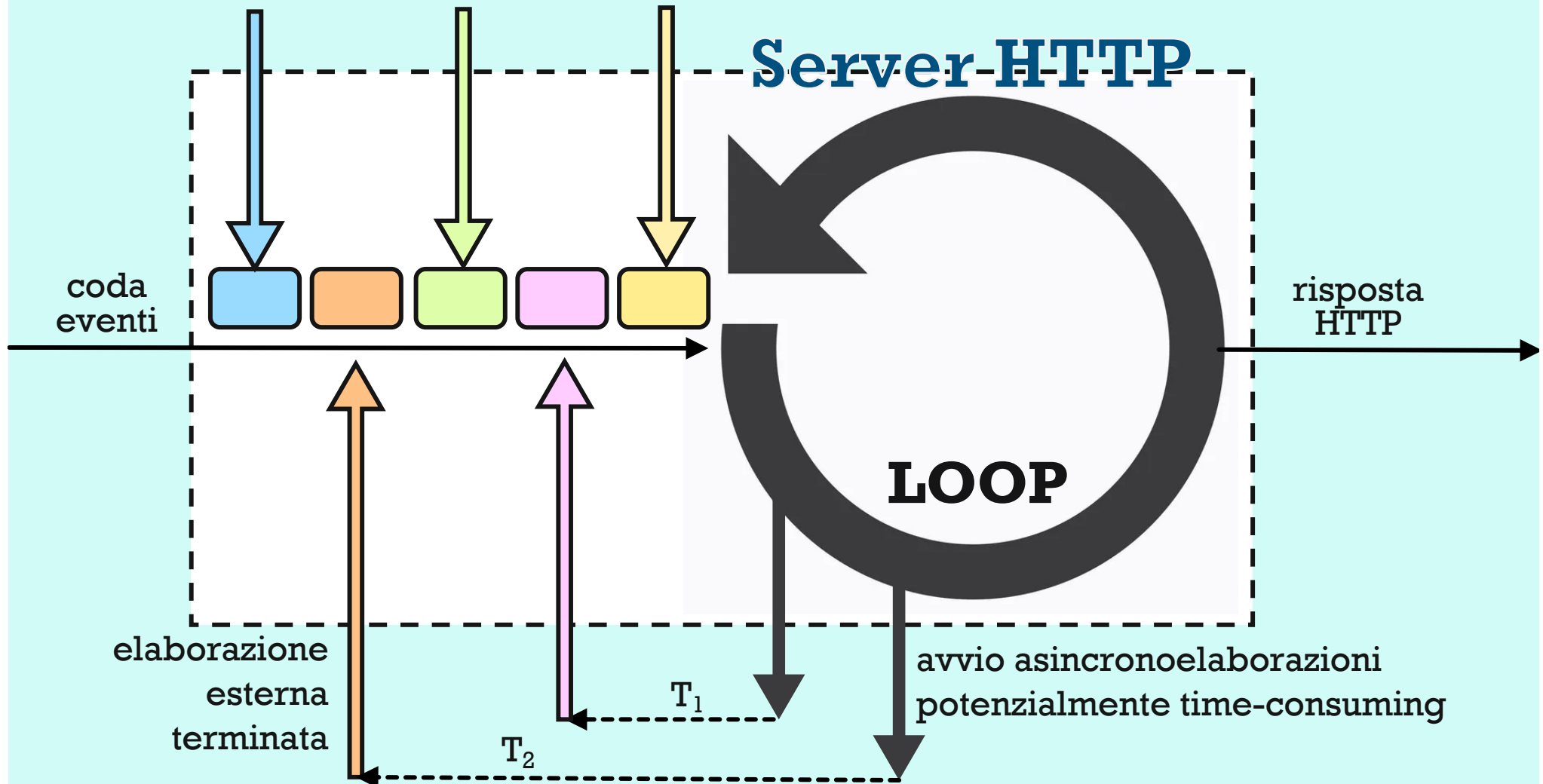
Core logic



HTTP Server

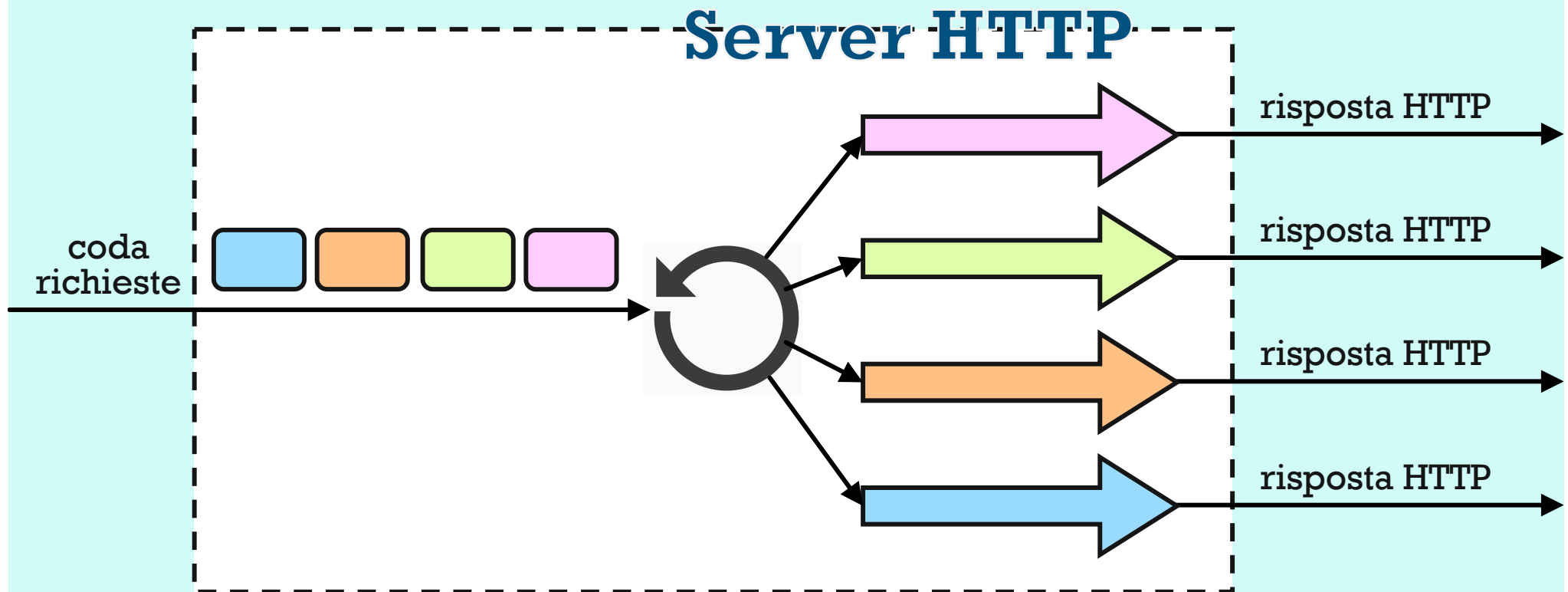
gestione **single-thread** con **asynchronous event loop**

Eventi di **arrivo di una nuova richiesta**



HTTP Server

gestione multi-thread



Server HTTP

Un server HTTP è un'**applicazione** che viene eseguita su una macchina detta **host** e sfrutta (fra le altre cose) i servizi **TCP/IP** del **sistema operativo** per ricevere **richieste HTTP** e rispondere ad esse.

Dall'esterno, è possibile rivolgersi all'**host** usando l'**indirizzo IP** ad esso assegnato quando è stato inserito in rete, o lo **hostname** corrispondente a tale IP. Per contattare uno specifico servizio di livello applicativo (come ad esempio l'HTTP Server) è necessario conoscere la porta su cui esso è in esecuzione (indirizzo IP o hostname qualificano l'host, sul quale però potrebbero essere in esecuzione plurimi servizi).

La maggior parte degli HTTP Server è in esecuzione sulla porta di default 80 (o sulla porta 443 se stiamo usando la variante sicura HTTPS); in tal caso non è necessario esplicitare la porta nella richiesta HTTP, in quanto verrà utilizzata quella di default.

Le cose cambiano un po' in fase di sviluppo e test di un'applicazione: scambiamo comunicazioni HTTP fra servizi in esecuzione su uno stesso computer. In questo caso, l'indirizzo IP è **127.0.0.1** e lo hostname è **localhost**. Lo strato TCP/IP del sistema operativo li conosce e sa che quando vengono utilizzati è perché il destinatario delle richieste coincide con il mittente.

L'unico modo per differenziare i diversi servizi in esecuzione resta dunque la **porta**; ecco perché capita spesso di avere più servizi, spesso addirittura più HTTP Server, in esecuzione su porte diverse.

Il compito di un Server HTTP, descritto in breve, è **restare in ascolto** sulla porta a lui assegnata e, quando arriva una richiesta, elaborarla e fornire una risposta.

Server HTTP – gestione delle richieste - I

Un Server HTTP deve gestire (almeno ipoteticamente) una grande quantità di richieste «in parallelo».

Le richieste giungono in una coda e vengono prese in considerazione nell'ordine di arrivo. Per ogni richiesta, l'elaborazione della risposta richiede una quantità di tempo **T** potenzialmente superiore all'intervallo che intercorre tra l'arrivo di due richieste. In altre parole, è ragionevole assumere che la coda sarà molto raramente vuota. Non si può pensare dunque che la coda sia gestita in modo completamente sequenziale e che una richiesta, per essere presa in carico, debba aspettare che tutte le richieste che la precedono in coda siano evase.

Sarebbe come se un laboratorio di analisi mediche non permettesse a un paziente di fare l'accettazione fino a che tutti coloro che lo precedono non hanno completato gli esami e non sono usciti dalla struttura. Se uno di loro deve fare un'esame particolarmente lungo, chi viene dopo di lui è costretto ad aspettare che lui abbia terminato, pur necessitando magari di un servizio molto più breve ed erogabile in parallelo. È sia uno spreco di risorse per chi eroga il servizio (personale, macchinari e spazi inutilizzati) che uno spreco di tempo per chi lo riceve.

Server HTTP – gestione delle richieste - II

Modello single-thread asynchronous event loop

In questo modello le richieste sono effettivamente prese in carico in modo sequenziale; il Server HTTP implementa infatti un loop a thread singolo. Questo implica che qualsiasi elaborazione necessaria per gestire una richiesta debba avvenire in modo asincrono:

- Le elaborazioni di I/O (verso il file-system, altre applicazioni/servizi quale ad es un DBMS, ecc) sono naturalmente asincrone, perché sono eseguite tramite il sistema operativo e non dal processo in cui è eseguito il Server HTTP
- Elaborazioni «CPU-intensive» devono essere gestite generando un thread separato in cui eseguirle

Nell'esempio del laboratorio di analisi, possiamo immaginare che al banco dell'accettazione ci sia una sola persona. Tuttavia questa persona si limita a esaminare la richiesta, capire di cosa si tratta, e redirigerla alla prossima operazione.

Se si tratta di un'operazione per cui l'utente deve solo aspettare (ad esempio, la stampa di un referto), la persona all'accettazione dà il via alla stampa e chiede all'utente di attendere fino a che non è terminata. Poi prende in carico la prossima richiesta.

Se si tratta di un'operazione che richiede di svolgere delle attività potenzialmente lunghe, la persona all'accettazione chiama un/a collega che prende in carico l'utente, e passa, anche in questo caso, alla prossima richiesta.

Server HTTP con questo modello (esempi): **Nginx**, qualsiasi HTTP Server basato su **Node.js**

Server HTTP – gestione delle richieste - III

Modello multi-thread

In questo modello ad ogni richiesta che arriva il Server HTTP genera un thread separato e dedicato per ciascuna richiesta, che la prende effettivamente in carico, e al proprio interno può gestire la richiesta anche volendo in modo sincrono.

Nell'esempio del laboratorio di analisi, questo corrisponde all'idea che, ogni volta che un utente entra nella sala di accettazione, appare un nuovo sportello con dietro una persona che si dedica interamente al nuovo arrivato, fino a che non ha terminato ciò che deve fare ed esce dall'edificio.

Server HTTP con questo modello (esempi): **Apache Web**, **Apache Tomcat**

A un esame superficiale, questo secondo modello potrebbe apparire molto più costoso in termini di risorse ma anche molto più efficiente del precedente nei confronti degli utenti.

In realtà non è davvero così: se in un primo momento questo secondo modello pareva l'unico praticabile, negli ultimi due decenni sono emersi Server HTTP basati sul primo modello che si sono rivelati persino più efficienti (il Server HTTP **Nginx** è praticamente nato al solo scopo di dimostrare che ciò era possibile!). Uno dei vantaggi del modello a single-thread è che ha performance **più prevedibili** sotto alto carico. Spesso è meglio una performance leggermente peggiore ma appunto più prevedibile: si pensi all'attesa alla fermata dell'autobus!

Con le attuali ottimizzazioni, possiamo dire che i due approcci sono comparabili sul piano delle risorse utilizzate e dell'efficienza di gestione delle richieste. È ciononostante importante sapere su quale modello si basa il Server HTTP sul quale si intende «montare» la propria applicazione server-side.

HTTP Server vs Web Server

Qual è, se c'è, la differenza?

Il **Web** è un **servizio** specifico, realizzato tramite il protocollo HTTP

Un server è un **Web Server** se effettivamente realizza quello che abbiamo chiamato il «core business», ossia fornire risorse statiche e dinamiche che contribuiscono al World Wide Web.

- per essere propriamente definito Web Server deve fornire risorse in modo sufficientemente «general purpose» anche qualora ritagliato sugli scopi di un'ente o un'azienda.
- di solito di un applicativo si dice che «fa da Web Server» non che «è un Web Server» perché il fatto che lo sia o meno dipende da come viene usato
- in linea di principio, nulla vieta di avere un Web Server che **non** è basato sul protocollo HTTP (anche se poi bisognerebbe anche avere un browser corrispondente...)

HTTP è un elemento infrastrutturale che definisce la modalità di comunicazione client/server

Quando diciamo **Server HTTP** ci riferiamo al protocollo con cui comunica, non agli scopi per i quali è utilizzato

- non è detto che un Server HTTP sia utilizzato come Web Server; ad esempio potrebbe essere usato semplicemente per esporre i dati di un DB, o soltanto come reverse proxy.
- per gli stessi motivi, un'applicazione server-side a sé stante che espone un'API HTTP non è certo un Server Web! tuttavia contiene necessariamente un Server HTTP quantomeno per ricevere le richieste che la riguardano.

Web/HTTP Server

Principali funzioni

Fornire **contenuti statici** e **dinamici** tramite protocollo HTTP

Gestione di **host virtuali**

Autorizzazione e **autenticazione**

Caching di richieste e relative risposte

Riduzione di banda (**bandwidth throttling**)

Riscrittura delle URL (**rewrite engine**)

Logging di informazioni sugli scambi HTTP per scopi statistici e di security

Funzioni di un Server Web/Http – I – Il Web

Il protocollo HTTP è nato per inviare documenti ipertestuali, e, in sostanza, per realizzare il World Wide Web. Pur con l'ampliamento del concetto di Web e del tipo di risorse che possono essere condivise tramite esso, la funzione più comune (anche se non l'unica) di un Server Web resta quella di fornire contenuti statici e dinamici; quest'ultima possibilità include dunque anche la possibilità per gli utenti di inviare dati al server al fine di «pubblicarli» verso altri utenti (la base, potremmo dire, di quello che è diventato noto come Web 2.0) e le vere e proprie Applicazioni Web, che come abbiamo visto si basano sullo scambio di risorse fra client/browser e server Web.

Le risorse scambiate sono, come abbiamo visto nello studiare il front-end, principalmente dati in formato JSON. Tuttavia ricordiamo sempre che un'applicazione client-side ha sempre come punto di ingresso almeno una pagina HTML con relativo CSS, e uno o più file contenenti il codice JS dell'applicazione. Quindi l'HTTP server che «pubblica» l'applicazione deve fornire queste risorse, più eventuali immagini o media usati dall'applicazione stessa. Queste risorse (l'HTML, il CSS, il JS, i file multimediali) possono essere sia statiche che dinamiche; quasi sempre un minimo di gestione di risorse statiche è necessario che il Server HTTP lo implementi, ma non è obbligatorio.

(*) il fatto che il Web sia il «core business», cioè l'impiego per cui il protocollo HTTP e i relativi server sono stati progettati, non significa che tutti gli HTTP Server espletino queste funzionalità. Ci sono HTTP Server che hanno altri scopi; giacché li perseguono basandosi sul protocollo HTTP, ricadono a pieno titolo nella categoria.

Funzioni di un Server Web/Http – II – Virtual Hosting

Talvolta un Server Web ospita più **host**, ciascuno con il proprio **hostname**. Nella maggior parte dei casi^(*) un HTTP Server ha un **unico indirizzo IP**. Dunque tutti gli **hostname virtuali** saranno mappati dal DNS sul medesimo IP. Per capire qual è la risorsa da fornire, il Server Web dovrà esaminare la richiesta HTTP per stabilire qual era lo **hostname** utilizzato dal mittente.

Un'alternativa è utilizzare il **numero di porta**: ogni **hostname** è mappato su un numero di porta differente. Tuttavia questo può generare dei problemi per via del fatto che il DNS **non mappa** la porta, e il browser tipicamente usa la porta di default.

Perché questa strategia funzioni bisogna spesso utilizzare il **reverse proxying** (alcuni Server HTTP possono essere usati a questo scopo, alcuni, come Nginx, vengono **principalmente** usati per questo scopo). Cosa fa un **reverse proxy** (per HTTP)? È un Server HTTP che riceve le richieste HTTP (ad es., sulla porta di default), e in base al loro contenuto le reindirizza altrove (ad esempio, al Server Web principale usando il numero di porta abbinato allo hostname contenuto nella richiesta). Il **reverse proxy** è molto utilizzato anche per gestire siti o applicazioni Web i cui contenuti e servizi sono in realtà erogati da molteplici Server Web; in questo caso, la richiesta specifica sempre il medesimo hostname, ma sarà il reverse proxy che, sulla base del pathname, redirige a un Server Web o a un altro.

(*) all'interno di una sottorete locale è perfettamente possibile usare un indirizzo IP come alias di un altro; in quel caso ogni **host virtuale** avrà un proprio **indirizzo IP** (corrispondente al relativo **hostname**) e il Server Web potrà disambiguare le richieste basandosi su di esso. Questa strategia è meno efficiente pertanto si tende, se appena possibile, a scegliere di mappare più hostname sullo stesso IP.

Funzioni di un Server Web/Http – III – Autorizzazione e autenticazione

Una delle funzionalità che molti Server Web forniscono è quella di permettere o vietare l'accesso ad alcuni **pathname/risorse** sulla base dell'IP del mittente. Si pensi ad esempio a una rete aziendale: alcune risorse saranno disponibili solo a computer **interni** alla rete aziendale stessa, altre (ad esempio il sito web aziendale) saranno disponibili pubblicamente.

In questa circostanza, se i dipendenti in smart working devono poter accedere alle risorse interne, sarà necessario che installino una **VPN (virtual private network)** che permette di «presentarsi» all'esterno con un diverso IP, per simulare l'appartenenza ad una rete locale, ed essere dunque ammessi alle aree riservate. Le VPN operano in modo legale: per assumere un IP appartenente a una rete locale, bisogna installare un certificato fornito dal proprietario della rete locale stessa.

Cosa diversa è l'**autenticazione**: anche spesso autorizzazione e autenticazione sono usate sinergicamente, per proteggere contenuti privati, non sono la stessa cosa. L'**autenticazione** ha a che vedere con il riconoscere l'identità dell'utente che sta richiedendo una risorsa. L'**autorizzazione** ha a che vedere con il permettere o vietare l'accesso a una risorsa; questa scelta può dipendere da molti fattori e l'identità dell'utente è solo uno di essi.

Funzioni di un Server Web/Http – IV – Caching

Un Server Web può effettuare il **caching** di risposte HTTP associate alla relativa richiesta.

L'obiettivo è evitare, a fronte della medesima richiesta, di reperire di nuovo la stessa risorsa. Questo vale a maggior ragione se la risorsa è **dinamicamente calcolata** con un'elaborazione costosa, o quando in realtà la risorsa deve essere richiesta ad un **altro** server (per esempio in caso di reverse proxying; tipicamente un reverse proxy effettua anche il caching). L'obiettivo del **caching** è velocizzare la risposta e diminuire il carico del server stesso.

Naturalmente soprattutto in caso di risorse dinamiche (ma anche in caso di risorse statiche) non si può pensare che la risposta alla richiesta sia la medesima in eterno: l'uso del meccanismo di caching, e la validità e scadenza della eventuale cache, possono essere negoziate fra client e server tramite il protocollo HTTP, utilizzando gli opportuni header.

Funzioni di un Server Web/Http – V – Bandwith Throttling

Quando un Server Web eroga dati che possono consumare significativamente l'ampiezza di banda in uscita, può ricorrere al **bandwith throttling** (letteralmente «strozzatura della banda»).

In pratica l'ampiezza di banda occupata da una singola risposta viene ridotta artificialmente, per permettere di suddividere la banda disponibile fra più stream paralleli in uscita.

Possiamo ricorrere all'analogia di più rubinetti che immettono acqua in una tubatura più grossa: se vogliamo che tutti i rubinetti possano far uscire la stessa quantità di acqua senza intasare il tubo principale, dobbiamo regolare l'apertura di ciascuno.

Come esempio possiamo considerare un server che emetta video streaming in alta qualità. Se un utente richiede un video, lo streaming inizierà, e occuperà tutta la banda disponibile. Quando arriva un secondo utente, se il server non riduce artificialmente la banda occupata dal primo, il secondo stream andrà a singhiozzo fino a che il primo non è terminato.

Funzioni di un Server Web/Http – VI – Rewrite Engine

Non sempre la URL che viene richiesta al Server Web dal client corrisponde al pathname che il server utilizza internamente per reperire la risorsa. Questo può essere un effetto voluto: magari le URL interne del server sono lunghe e complesse, con querystring lunghissime e pathname molto articolati per gestire API complicate di applicazioni multiple; si vuole invece che i client possano utilizzare URL «pulite» e evocative di quello che si sta richiedendo. Per gestire queste situazioni, alcuni Server Web includono un «motore di riscrittura», ossia una componente che applica regole sintattiche per tradurre le URL dalla versione «clean» alla versione «raw». In alcuni casi è anche possibile specificare regole diverse da un sito (o applicazione) all'altro.

Ad esempio, se si dispone di uno spazio web per il proprio sito o la propria applicazione su un Server HTTP Apache Web, è possibile specificare le regole di riscrittura in un file denominato **.htaccess**. Naturalmente bisogna sapere come scriverle...

Quando si adotta (ad esempio in React) un meccanismo di **routing lato client**, il Rewrite Engine diventa molto importante. Nel **routing lato client** il framework sostituisce la navigazione standard del browser con un meccanismo proprio. Quando l'utente clicca su un link, il **router** legge la URL di destinazione, e se capisce che si riferisce a una «location» all'interno dell'applicazione, invece di effettuare una vera e propria navigazione (che porterebbe a un reset dell'applicazione) va semplicemente a configurare la UI in modo che essa mostri la «location» richiesta dall'utente.

Tuttavia, questo funziona se l'utente **clicca** su un collegamento, o comunque lo attiva dall'interno dell'applicazione. Ma cosa succede se l'utente scrive la URL nella barra del browser? Il browser fa partire la richiesta al server. L'azione corretta del Server Web sarebbe di reindirizzare la richiesta sulla pagina dell'applicazione, ma perché ciò accada è necessario appunto l'intervento del Rewrite Engine. In fase **dev** questo problema non si pone perché usiamo due Server HTTP diversi per servire le due applicazioni client-side e server-side, dunque il Server HTTP che fornisce l'applicazione lato client è un server interno dell'ambiente di sviluppo che è interamente dedicato alla client-side app per cui sa benissimo cosa fare quando riceve la URL incriminata.

Comunicazione con la server-side app

server-side API

Lo scopo della **API** è esprimere le funzionalità offerte dalla app lato server.

- Come le si richiede tramite **HTTPRequest**
- Che tipo di **HTTPResponse** ci si può aspettare in risposta, e quali altri effetti ha l'invocazione della funzionalità

HTTPRequest

METHOD

PATHNAME/ROUTE

BODY

HTTPResponse

BODY

Server-Side API - I

Una server-side app (ossia, ai nostri fini, un'applicazione che risponde a richieste HTTP, ma lo fa scambiando principalmente **dati** in formato JSON) **espone un accesso al Data Model**.

Il Data Model è più naturalmente gestito lato server, perché è il server ad avere la possibilità di rendere persistenti i dati, comunicando con un database.(*). Si dice in generale che l'applicazione **server-side** è la **source of truth** per il data model.

Applicazioni più complesse possono costruire sopra al modello dati una logica di business più articolata, che elabora i dati del DB e costruisce su di essi dei servizi «live», che forniscono funzionalità complesse, calcolate combinando i dati grezzi presenti nel DB con informazioni circostanziali o provenienti da terze parti.

Quindi possiamo vedere la cose in questo modo: l'app lato client **usa le funzionalità** dell'app lato server. Abbiamo visto che perché un'applicazione possa usare le funzionalità di un altro software (un'altra app, una libreria, un servizio online...) quest'ultimo deve dichiarare la propria API = Application Programming Interface.

In astratto una API è fatta di **funzionalità** che possono essere invocate; per ciascuna l'API dovrà dichiarare

- Come si invoca proprio quella funzionalità e non un'altra
- Quali input devono e/o possono essere forniti
- Quali sono gli effetti attesi e in particolare quali output vengono restituiti al chiamante

(*) non è impossibile per un'applicazione client-side rendere i dati persistenti ma si tratterà comunque di una persistenza limitata ai dati del singolo utente sul un singolo dispositivo; questo rende molto difficile la condivisione delle informazioni fra utenti diversi e fra dispositivi diversi, dunque si tratta di una persistenza un po' limitata.

Server-Side API - II

Dunque, come si descrive un'API «costruita» sopra il protocollo HTTP?

Vediamo un elemento per volta.

Qualificare le funzionalità

Per specificare alla server-side app quale funzionalità vogliamo richiedere, dobbiamo usare ciò che la HTTPRequest ci mette a disposizione; in particolare si useranno:

- Il **pathname** della URL: ogni funzionalità è resa disponibile su un diverso pathname. Un pathname si compone di più segmenti separati da slash (/); una particolare sequenza di segmenti va a identificare una ben precisa funzionalità, ad esempio **/bookshelf/books** o **/bookshelf/collections** o ancora **/requests/expired** o anche **/login**. In questo contesto si parla anche di **route** proprio perché la sequenza di segmenti **instrada** verso una ben precisa funzionalità (**attenzione**: il concetto di **route** è applicabile anche lato client, se ne avremo la possibilità ne parleremo in seguito).
- Il **method**: è possibile avere più funzionalità disponibili sullo stesso pathname a patto che usino un **method** diverso. In questo corso vediamo come method soltanto GET e POST. In linea di massima si usa GET per le funzionalità che (per lo più) **forniscono** dati o risorse dal server verso il client, e POST per le funzionalità che (per lo più) permettono al client di inserire nuovi dati nel Data Model. Ad esempio, **GET /bookshelf/collections** potrà ottenere l'elenco delle raccolte di libri, mentre **POST /bookshelf/collections** potrà aggiungere una nuova raccolta. Si tratta di una semplificazione; non tutte le funzionalità cadono in una di queste categorie. In quel caso si sceglierà quale **method** usare in base all'input che la funzionalità richiede, e per il quale rimandiamo alla discussione seguente.

Server-Side API - III

Esprimere gli input

Una funzionalità server-side che richieda degli input dovrà specificare come essi devono essere espressi all'interno della richiesta HTTP. Potrà farlo utilizzando tre diversi aspetti della HTTPRequest:

- Il **body** della Request, che però è disponibile solo per le richieste di tipo POST(*); tipicamente si usa per funzionalità che necessitano di un input «corposo» e strutturato, come ad esempio un array di valori o un oggetto. D'altra parte sono tipicamente le richieste di tipo POST a necessitare di questo tipo di input.
- La **query string**. Piccolo ripasso dalla prima parte del corso:

Query string

Si tratta di un testo opzionale che aggiunge ulteriori informazioni alla richiesta, sotto forma di **coppie chiave (c)/valore (v)**. La query string inizia sempre con il carattere ? seguito dalle coppie **c=v** separate, se ce ne fosse più di una, da un &. Quindi ad esempio una query string può avere la forma:

`?min=5&max=100&sort=alphabetical`

Ciascuna coppia chiave/valore costituisce un input.

Questo modo di richiedere degli input può essere utilizzato sia in caso di GET che di POST. Anche se abbiamo visto che teoricamente interi oggetti o array possono essere codificati come stringhe tramite JSON, anzi, qualunque file può esser codificato come una stringa e dunque passati come valori in una query string (e in effetti non è infrequente vedere query string lunghissime!), è più «elegante» usare la query string quando gli input sono valori semplici o comunque poco strutturati.

(*) questo perché ci stiamo restringendo alle sole GET e POST, naturalmente ci sono altri **method** che prevedono un **body** nella Request.

Server-Side API - III

Esprimere gli input (continua)

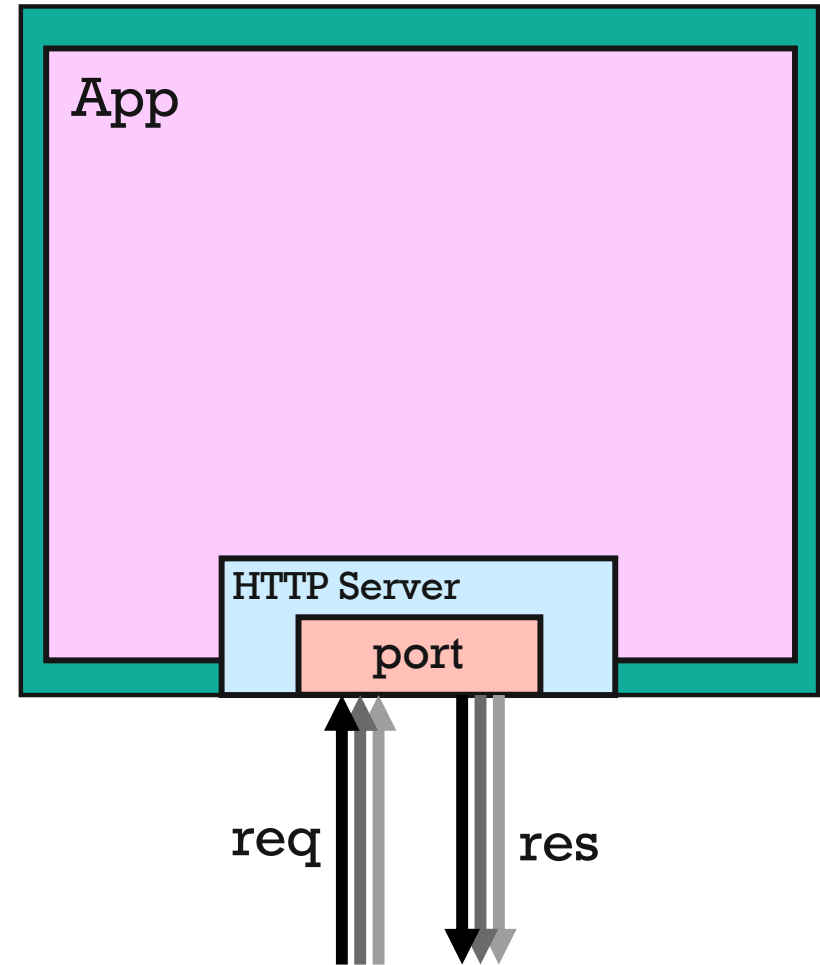
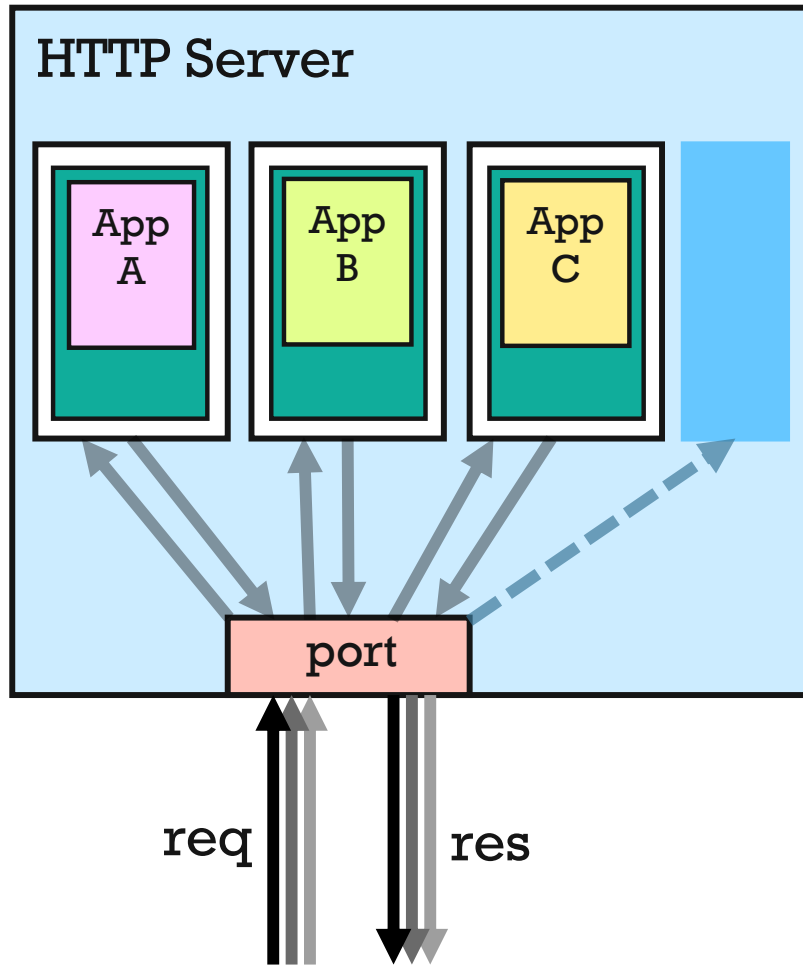
- La **route**, o per meglio dire, alcuni **segmenti** della route, che, quando così utilizzata, viene detta **route parametrica**. Quello che succede è che alcuni segmenti del pathname sono variabili e corrispondono a un determinato input. Ad esempio, la server-side API potrebbe dichiarare che per ottenere i dati di un libro si può usare la route **GET /bookshelf/books/:id** dove il segmento **:id** è di volta in volta sostituito dall'id del libro di cui si vogliono i dati. Dunque l'app client-side potrà effettuare la sua HTTPRequest su **/bookshelf/books/12** se vuole i dati del libro con **id** pari a 12, oppure su **/bookshelf/books/5** se vuole i dati del libro con **id** pari a 5. Questa modalità di specifica degli input viene tipicamente utilizzata per input che identificano una specifica risorsa (id, nomi, ecc). Ricordiamo infatti che la URL nel suo complesso dovrebbe essere un «Resource Locator», come dice il suo stesso acronimo. Inoltre è una modalità che funziona se gli input in questione sono numeri o brevi stringhe.

Dichiarare gli effetti attesi e le informazioni inviate in risposta

Questa componente dell'espressione di una funzionalità offerta tramite API è per certi versi la più semplice: la server-side API dovrà dichiarare (tramite opportuna documentazione), per ciascuna funzionalità offerta, in cosa consiste effettivamente quella funzionalità, in particolare:

- Le modifiche al Data Model conseguenti all'esecuzione della funzionalità e in dipendenza degli input ricevuti.
- Come sono fatti i risultati dell'elaborazione che vengono eventualmente reinviati al chiamante tramite il **body** della HTTPResponse.

Server HTTP e applicazione server-side come si relazionano?



Server HTTP ed applicazioni server-side - I

Un'applicazione server-side è un **processo** che resta in esecuzione, in attesa di dover processare una richiesta HTTP ed elaborare una risposta. Qual è la sua relazione con l'HTTP server? L'applicazione **usa** l'HTTP Server? O l'applicazione è un HTTP Server?

Vediamo i due casi più emblematici (sono possibili architetture più complesse, ma tutte bene o male riconducibili a queste due macro situazioni).

a) il Server HTTP «contiene» una o più app

In questo caso il Server HTTP esiste per sé, è un'applicativo che è stato installato e che in frequentemente fa altre cose oltre a fornire accesso alle applicazioni server-side. Spesso è un Server Web, e fra le risorse web che fornisce c'è anche l'applicazione client-side (ossia, il file HTML/CSS/JS che la costituiscono).

I dettagli di «come» le applicazioni vengono concretamente messe in opera dipendono dallo specifico Server HTTP; in generale tuttavia possiamo dire che il Server HTTP contiene un motore di esecuzione (ad esempio, una Java Virtual Machine) che tiene «accese» le applicazioni. Tipicamente in questo caso viene creato un **ambiente di esecuzione privato** per ogni app (il **Web Container**) con una propria istanza del motore di esecuzione, e viene avviato un thread separato di esecuzione per ciascuna app. Siamo dunque nel modello **multi-threaded**. Ogni app dovrà dichiarare quali sono i pathname/route sui quali si incarica di rispondere, e questa informazione dovrà venire registrata dall'HTTP Server in un'apposita **tabella di routing**. Quando una richiesta HTTP **req** arriva al Server HTTP sulla porta di default, tramite la tabella di routing esso è in grado di inoltrare la richiesta all'app destinataria, invocandone opportune funzioni. Tali funzioni dovranno restituire al Server HTTP la risposta HTTP **res**, perché possa inviarla al client.

Server HTTP ed applicazioni server-side - II

b) l'app «contiene» il server HTTP

In questo caso il punto di ingresso è il **motore di esecuzione** del linguaggio di programmazione, all'interno del quale l'applicazione si mette in ascolto su una porta in attesa di richieste HTTP. Tipicamente in questo caso le librerie del linguaggio di programmazione mettono a disposizione un **Server HTTP** di base che l'applicazione può istanziare e mettere in ascolto su una porta a sua scelta. Dunque possiamo dire che l'applicazione **contiene** il Server HTTP.

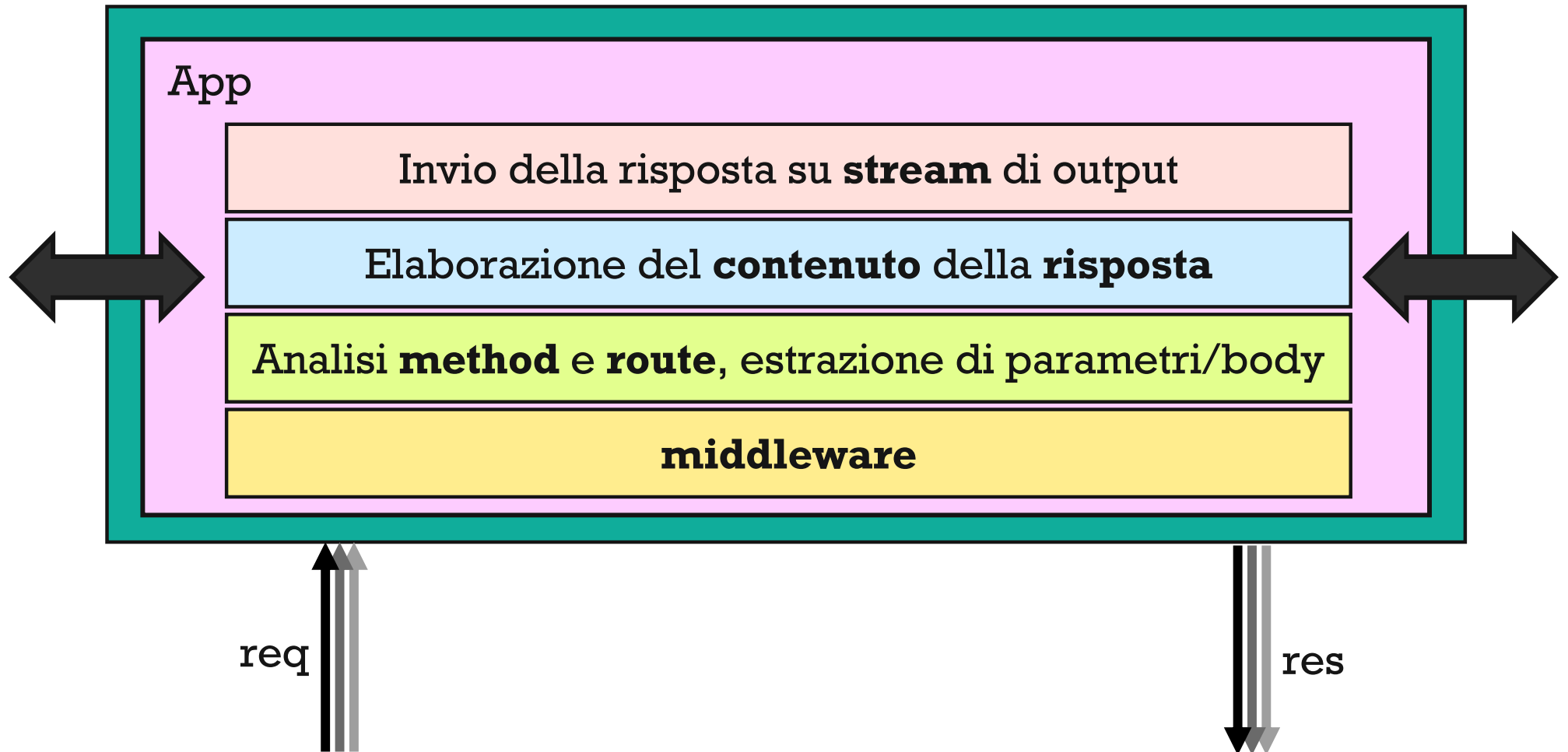
Anche in questo caso il Server HTTP interno all'applicazione «avverte» tramite un meccanismo event-driven quando arriva una richiesta HTTP, e la inoltra ad un'apposita funzione dell'app che resta in ascolto/attesa. La funzione in questione restituirà una risposta che il Server HTTP invia al client. Su questo aspetto i due modelli sono analoghi. La differenza principale è che nel caso (a) è il Server HTTP che ha il controllo e «lancia» l'applicazione; nel caso (b) è l'applicazione che ha il controllo e «lancia» il Server HTTP.

In questo modello la nostra applicazione è un Application Server basato su HTTP. Nulla impedisce che l'implementazione interna sia tale da permetterle di essere anche un Web Server, ma questo aspetto va esplicitamente incluso (così come tutte le altre eventuali funzionalità da Web Server menzionate precedentemente), sebbene ciò si possa spesso fare utilizzando opportune librerie o package.

Questo modello si abbina più facilmente con un Server HTTP **single-thread**. Ma, come abbiamo accennato, nulla impedisce di creare un'architettura con più app che rispondono su porte diverse, in abbinamento con un **reverse proxy** che ascolta sulla port di default e smista le richieste. In questo modo il modello (b) si avvicina un po' al modello (a).

L'app e il mondo che la circonda

L'ambiente di esecuzione



Caratteristiche generali di un'app server-side - I

Il mondo esterno all'app

Qualunque sia il modello di esecuzione dell'app server-side, essa comunica con il **mondo esterno** attraverso il suo ambiente di esecuzione, che dovrà offrirle opportune librerie per farlo.

Per «mondo esterno» intendiamo principalmente:

- il sistema operativo della «macchina» (reale o virtuale) in cui viene eseguita l'applicazione e/o il Server HTTP. in particolare il file system, che molte applicazioni usano per scopi di logging
- nel caso (a), informazioni di contesto di esecuzione fornite dal Server HTTP
- servizi o risorse di rete, tramite opportune connessioni di rete di livello application; in questa categoria ricade la connessione con un DataBase Management System di cui diremo di più in seguito

Elementi/Fasi di esecuzione dell'app

Middleware: Prima ancora che la richiesta HTTP venga presa in carico dall'app vera e propria, entra in gioco (se necessario) il cosiddetto **middleware**, costituito da una o più funzioni che effettua elaborazioni preliminari sulla richiesta HTTP. La tipica funzione di **middleware** prende in input una richiesta HTTP, opzionalmente la trasforma, e la «passa oltre», alla prossima funzione di middleware o all'applicazione, oppure eventualmente la blocca. In virtù di questa modalità, le funzioni di middleware sono anche dette **filtri**.

Il middleware può ad esempio respingere richieste **non autorizzate**, interpretare e trasformare il **body** o i **parametri** della richiesta, corredare la richiesta di **informazioni aggiuntive**, etc. È uno strato utile per quelle operazioni che dovrebbero essere eseguite sempre e comunque per ogni richiesta pervenuta.

Caratteristiche generali di un'app server-side – II

Elementi/Fasi di esecuzione dell'app (continua)

Individuazione della richiesta: abbiamo visto che l'API dell'applicazione server-side si basa sulla struttura della richiesta HTTP, in particolare è necessario analizzare la combinazione di **method** e **route** per determinare qual è la funzione invocata, ed estrarre l'input, sia esso sotto forma di **segmento di una route parametrica**, **elemento della querystring** o **body**.

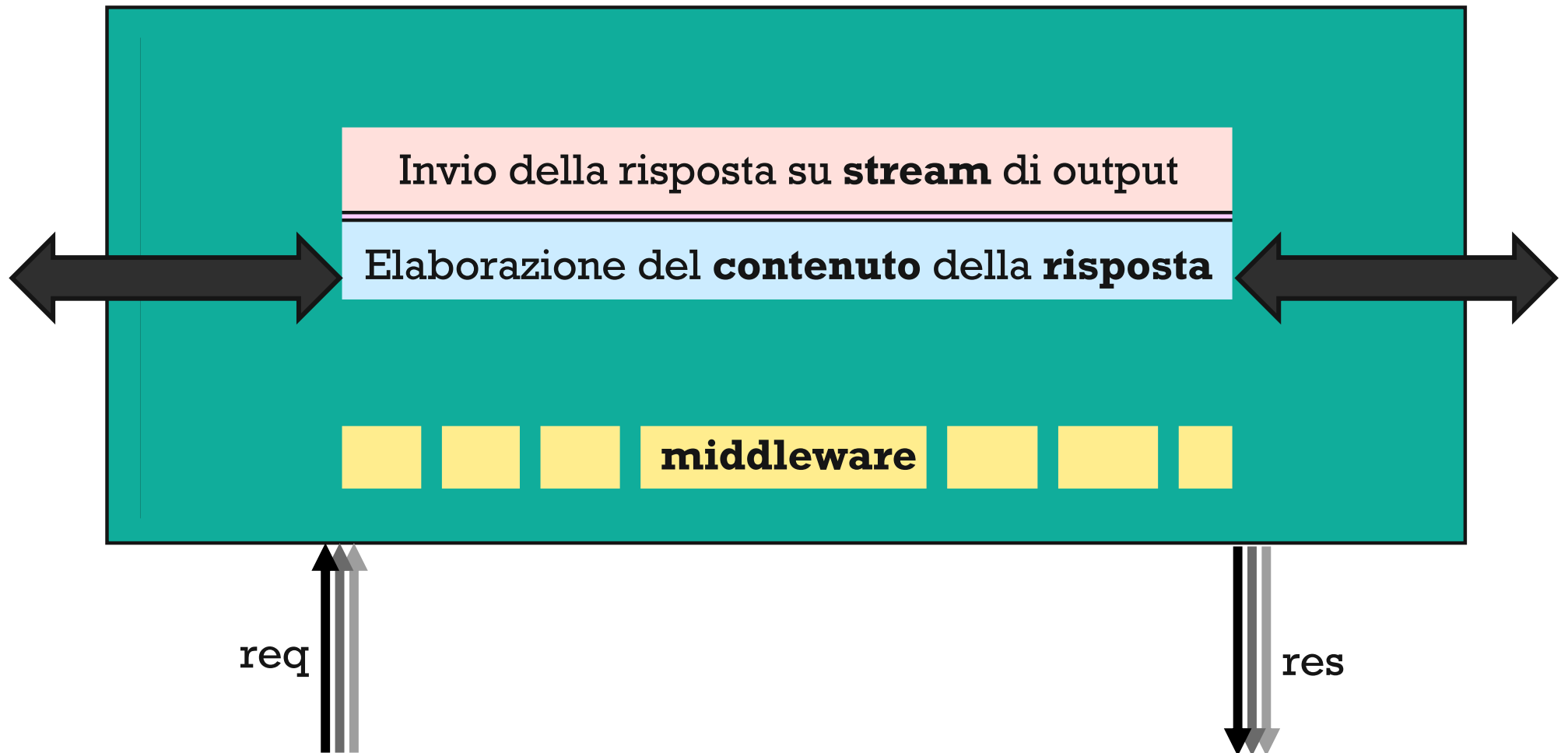
Elaborazione della risposta: questo è il «core» dell'applicazione server-side, il punto in cui essa esegue le proprie funzionalità. Qualunque sia la funzionalità richiesta, l'app dovrà elaborare una risposta, che potrà contenere i risultati dell'elaborazione o anche solo un acknowledgement di corretta esecuzione.

Output della risposta: una volta elaborata la risposta, è necessario inviarla (tramite la mediazione del Server HTTP) al client. La risposta viene **sempre** inviata in streaming; la comunicazione dal Server HTTP verso il client/browser è gestita come un buffer, per permettere l'invio di file anche di grandi dimensioni senza che ciò sia bloccante. Abbiamo già visto che anche dal punto di vista del client, i dati contenuti nella risposta arrivano in streaming; l'arrivo della risposta al client segna solo l'inizio del flusso di dati.

Errori? Naturalmente l'esecuzione dell'app deve prevedere la possibilità che ci siano errori nella richiesta, o perché vorrebbe compiere operazioni non autorizzate, o per inconsistenze nella formulazione della stessa, o parametri non accettabili. E naturalmente si possono verificare eccezioni/errori nell'elaborazione. In questo caso bisognerà inviare una risposta con status code appropriato (4xx o 5xx a seconda dei casi).

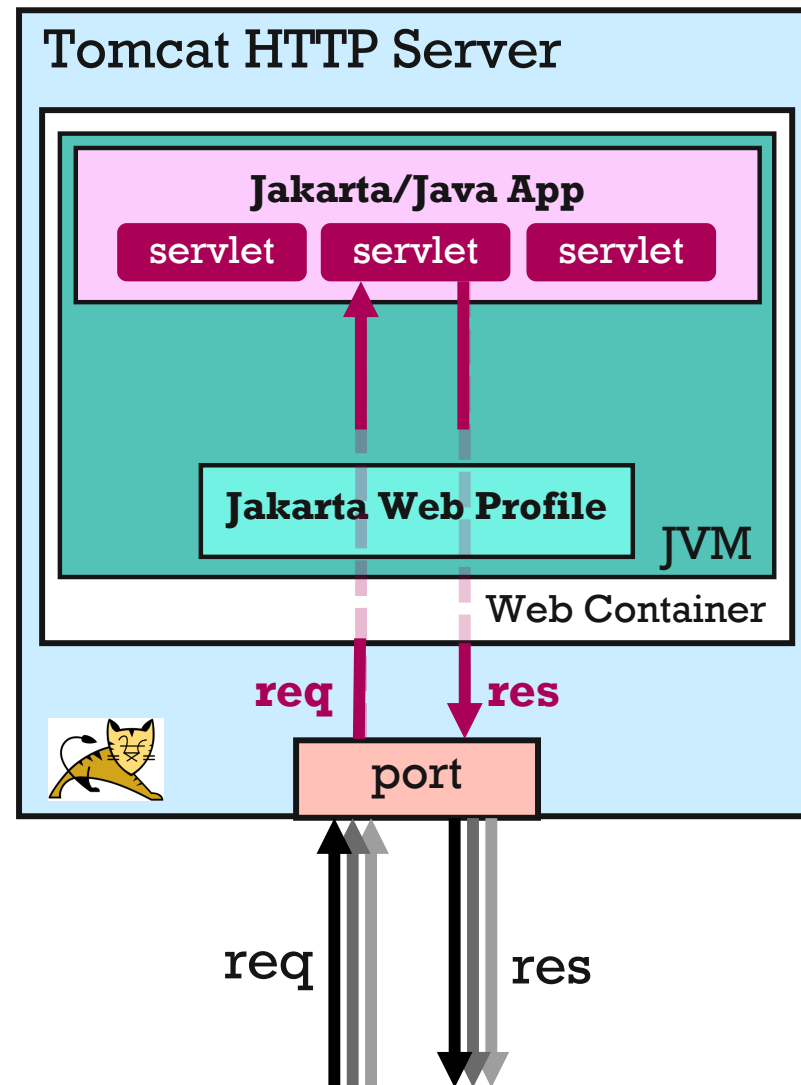
Framework server-side

Evitare i compiti ripetitivi



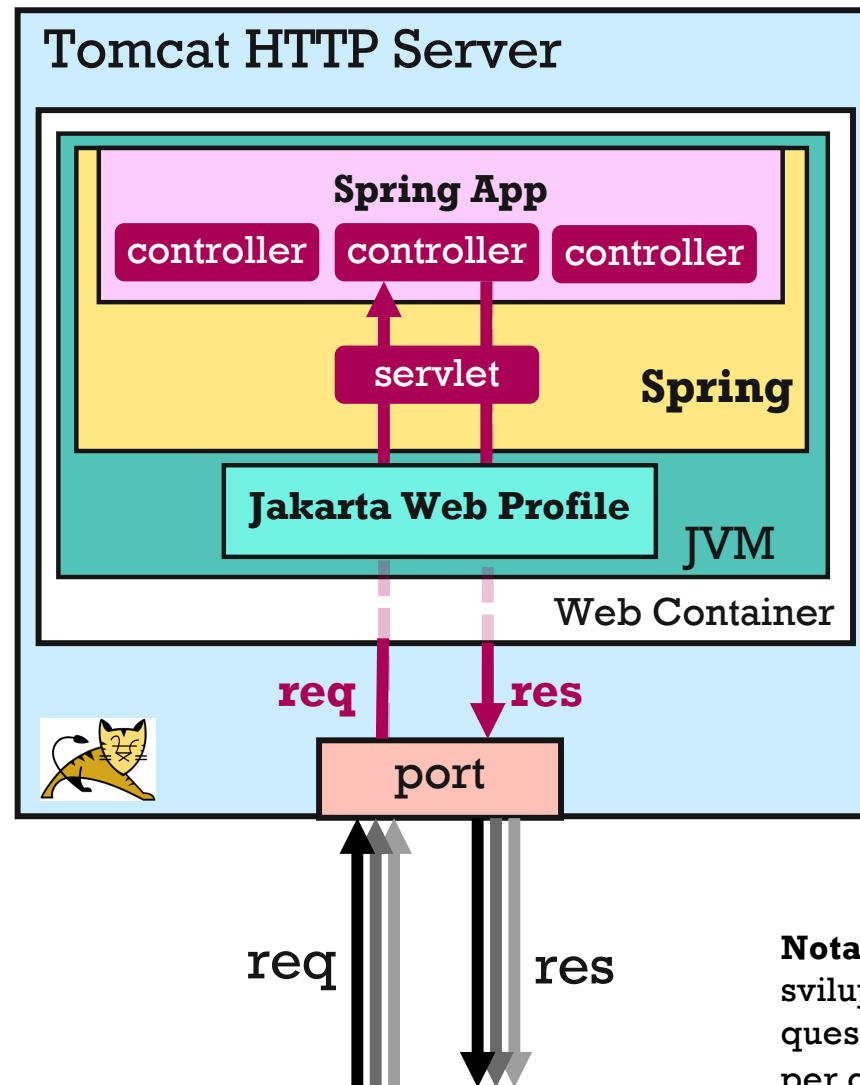
server-side app con Jakarta Web Profile

l'ambiente Java-based per lo sviluppo server-side



server-side app con Spring

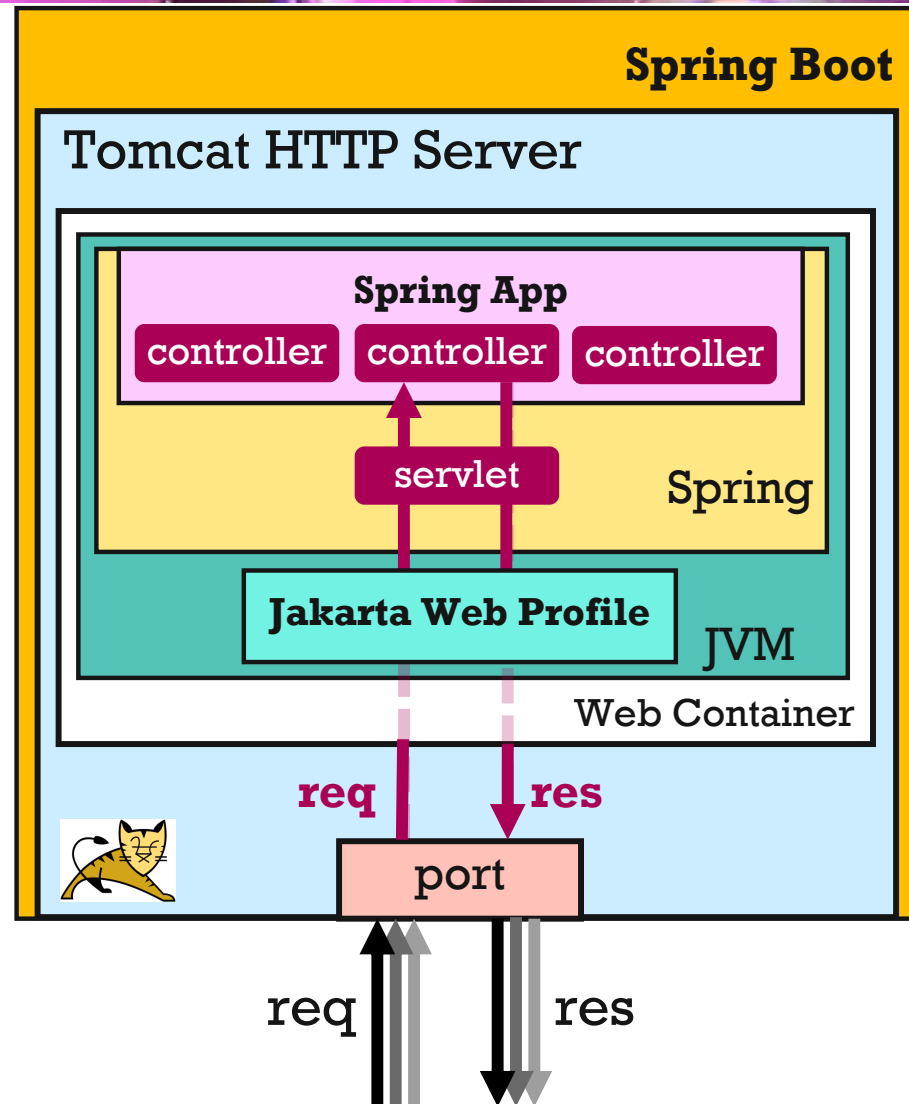
qual è la relazione fra Spring e Jakarta?



Nota: Spring contiene, ed offre agli sviluppatori, molto più che Jakarta EE; questa rappresentazione è semplificata per gli scopi di questo corso

server-side app con Spring-Boot

server-side standalone apps



Sviluppo applicazioni server-side con Java/Jakarta - I

Nelle applicazioni web basate sul Jakarta Web Profile si parte dal presupposto che sia il server HTTP, che esiste come applicativo indipendente, a incapsulare l'applicazione. Il server HTTP di riferimento per le app con Java/Jakarta è Apache Tomcat, che si basa sul modello **multi-thread**.

Per permettere l'esecuzione al proprio interno di applicazioni Java, Tomcat implementa la specifica Jakarta Web Profile che fa parte di Jakarta Enterprise Edition (precedentemente nota come Java Enterprise Edition).

Jakarta Enterprise Edition (Jakarta EE in breve) è un insieme di specifiche che estendono Java con librerie, componenti e API che supportano lo sviluppo di applicazioni commerciali, e che non sono incluse nel JDK standard.

Jakarta Web Profile è il sottoinsieme di specifiche e librerie orientato allo sviluppo di **applicazioni web**. Il runtime di Tomcat implementa le specifiche di Jakarta e fornisce le sue librerie, realizzando un ambiente di esecuzione per l'applicazione server-side, costituito da un insieme di **classi**, **interfacce** e **oggetti java** che possono essere utilizzati per realizzare le funzionalità previste dall'applicazione.

Sviluppo applicazioni server-side con Java/Jakarta - II

Il Jakarta Web Profile offre in particolare due strumenti principali:

- le **Servlet**, oggetti i cui metodi ricevono richieste HTTP, elaborano la risorsa richiesta, e la inviano in risposta. Lo sviluppatore realizza la sua applicazione realizzando le proprie **classi-Servlet**, che verranno istanziate da Tomcat nel momento in cui l'applicazione viene avviata. Ogni Servlet può specificare quali sono le **route** a cui risponde; sarà Tomcat a registrare questa informazione nella propria **tabella di routing** e ad inoltrare le richieste HTTP alle servlet opportune.
- le **JSP** (Jakarta Server Pages): pagine scritte in un linguaggio che intreccia HTML e frammenti di codice Java. A ciascuna di queste pagine Jakarta abbina una propria **Servlet** che, alla ricezione della richiesta, usa il contenuto della JSP per **generare una pagina HTML** da inviare al client. Con le JSP possiamo dunque realizzare un'applicazione Web **multi-pagina orientata al server**.

Nel seguito vedremo qualche dettaglio in più sul funzionamento delle **Servlet**, che utilizzeremo indirettamente in quanto sono alla base del funzionamento di **Spring**. Non vedremo invece le JSP in quanto abbiamo deciso di orientarci verso applicazioni **single-page** la cui architettura prevede due applicazioni indipendenti (una client-side e una server-side) che comunicano scambiando dati in formato JSON. In questo tipo di applicazioni la UI è costruita dal browser, non è generata dal server.

Prima di questo, vediamo brevemente qual è la relazione fra Jakarta e Spring.

Il ruolo di Spring nell'architettura complessiva

Ai fini del nostro corso, il ruolo di **Spring** è quello di evitare al programmatore la scrittura di **boilerplate code** (ossia codice ripetitivo deputato alla inizializzazione e gestione delle componenti di base del sistema, e non collegato in alcun modo alla logica dell'applicazione specifica)

Lo sviluppatore in Spring realizza dei **request handler**, fornendo al framework informazioni su

- a quali tipi di richieste HTTP ciascun handler risponde (ossia: a quale **method/route**)
- quali dettagli della richiesta HTTP gli servono per funzionare (ad es.: gli servono i **parametri** della query string, piuttosto che il **body** della request, piuttosto che i parametri ottenuti tramite segmenti parametrici della URL)

Il framework si incaricherà di invocare l'**handler** corretto e fornirgli le informazioni richieste.

Come vedremo meglio in seguito, gli handler in Spring sono raggruppati all'interno di classi dette «**controller**». All'interno di Spring (e lontano dall'occhio dello sviluppatore) ciascun **controller** corrisponde di fatto a una **servlet** di Jakarta.

Naturalmente **Spring** non è tutto qui: il framework fornisce moltissime altre funzionalità. Ad esempio, facilita enormemente l'accesso ai dati su DB. Ma una cosa alla volta...

Spring Boot

La piattaforma Spring è molto versatile e può essere configurata in una varietà di modi, e con un livello di dettaglio molto sottile. Questo vale sia per le librerie che si sceglie di includere, sia per la configurazione dell'esecuzione che dipende, fra le altre cose, da come è installato e configurato Tomcat.

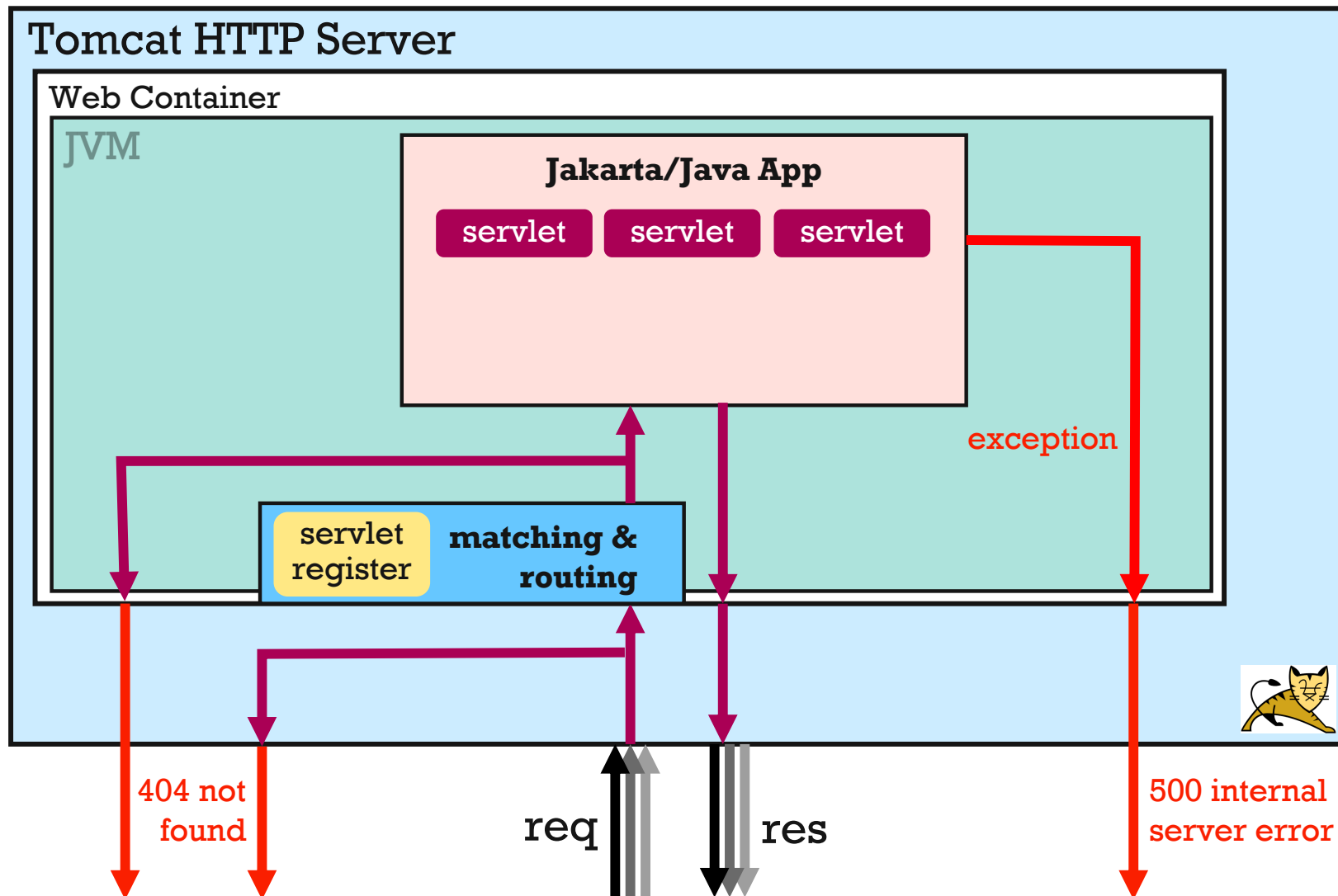
Spring Boot estende Spring adottando un approccio «dogmatico» rispetto ad alcune scelte architetturali e progettuali su cui Spring è di per sé laico. È proprio la «laicità» di Spring a rendere complessa la configurazione di ogni singolo progetto. Le scelte per certi versi imposte di Spring Boot d'altro canto semplificano, velocizzano e rendono più efficiente la realizzazione di applicazioni, a patto che siano compatibili con codeste scelte.

I punti principali che possiamo menzionare sono:

- Spring Boot offre dei **pacchetti di dipendenze standard** che evitano di andare a specificare una per una le librerie che si intende usare; naturalmente è poi sempre possibile andare ad aggiungere dipendenze custom
- Spring Boot **incapsula un'installazione di Tomcat**, e la gestisce internamente, dunque un'applicazione Spring Boot è di fatto un'applicazione stand-alone il cui **deploy** è grandemente semplificato dal fatto che non bisogna fare i conti con la configurazione pre-esistente dell'HTTP server.
- Spring Boot fornisce **modelli di configurazione predefiniti** per molte delle funzionalità di Spring, che sono adeguati per la maggior parte dei casi e risparmiano allo sviluppatore di farlo a mano.

Jakarta Web Profile

architettura di esecuzione di app basata su Servlet



Architettura di esecuzione di una Servlet-based app

Gli attori principali di questa architettura sono il Server HTTP (Tomcat), l'applicazione vera e propria (su uno stesso Server possono esserci una o più istanze della medesima applicazione) e il Web Container, che agisce da interfaccia fra il Server e le applicazioni, **implementando** la Jakarta Web Profile spec. e **gestendo il ciclo di vita dell'applicazione**, ossia **delle servlet** che la compongono.

Il Server HTTP riceve le richieste HTTP; il suo ruolo (rispetto alle Jakarta Web App installate al suo interno) è di valutare tramite il **pathname** se una richiesta

- deve andare verso una delle app (tipicamente ogni app è caratterizzata da un segmento iniziale di pathname), nel qual caso la passa al **Web Container**
- deve essere gestita altrimenti (ad es. servendo una risorsa statica)
- fa riferimento a una risorsa inesistente, nel qual caso risponde con **404 not found**

Il **Web Container** è responsabile di:

- **inizializzare** gli oggetti **servlet** all'avvio del Server stesso, o quando l'applicazione viene rilasciata e installata per la prima volta.
- gestire appunto le richieste HTTP per l'applicazione e invocare i **servizi** di una opportuna **Servlet**
anche se il pathname di una richiesta **punta** verso una data app, non è detto che l'app gestisca la specifica **route** indicata nella richiesta. È possibile che si generi a questo punto un **404 not found**
- gestire eventuali **eccezioni** sorte nell'app e in tal caso inviare in risposta un 500 internal server error
- **distruggere** gli oggetti **servlet** allo spegnimento del Server, o se l'applicazione viene disinstallata.

Servlet at a glance

Diamo un'occhiata a HelloServlet.java

```
package tweb.myfirstapp;

import java.io.*;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.*;

@WebServlet(name = "helloServlet", value = "/hello-servlet")
public class HelloServlet extends HttpServlet {

    private String message;

    public void init() { /* ... */ }

    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws IOException {
        /* ... */
    }

    public void destroy() { /* ... */ }
}
```

Un progetto con Spring Boot

Inizializzazione e gestione di progetto

Lo scheletro di progetto viene inizializzato tramite lo **SpringInitializr**

Specificate alcune opzioni di base, genera la struttura del progetto, le dipendenze di partenza e la configurazione iniziale.

- possiamo generare uno zip contenente il tutto a partire dall'interfaccia Web **start.spring.io** per poi scompattarlo e aprirlo in un qualsiasi **IDE**
- IntelliJ volendo fa il lavoro per noi: in fase di creazione di un nuovo progetto basterà scegliere il «generator» denominato **Spring Boot**
- Come dipendenza di partenza, specifichiamo per il momento solo **Spring Web**

In modo analogo a quanto visto per l'app client-side con **npm**, anche l'ambiente di sviluppo server-side usa un tool per la gestione del progetto e delle sue dipendenze: **maven**

- in Spring Boot è possibile scegliere a questo scopo fra **maven** e **gradle**; qui proponiamo **maven** perché è più lineare e semplice da utilizzare anche se meno flessibile. Per i nostri scopi è più che sufficiente!
- **maven** memorizza la configurazione attuale e le dipendenze di progetto nel file **pom.xml**

Un progetto con Spring Boot

Struttura del progetto

▼ demo /Volumes/dev/TWeb/SpringBoot/prove/demo

- > .idea
- > .mvn
- ▼ src
 - ▼ main
 - ▼ java
 - ▼ unito.tweb.demo
 - HelloController
 - MyApplication
 - resources
 - static
 - templates
 - application.properties
 - > test

.gitattributes
.gitignore
HELP.md

 - mvnw
 - mvnw.cmd
 - pom.xml

Sorgenti java

Configurazione di Spring Boot per questa specifica applicazione

Script di esecuzione di **maven** (Unix/Linux e Windows)

Configurazione di progetto e **dipendenze** via **maven**

Gestione dipendenze

Il file pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

versione di XML

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
```

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
xsi:schemaLocation=
```

link allo XML Schema in uso

```
"http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd"
>
```

```
<modelVersion>4.0.0</modelVersion>
```

```
<parent>
```

```
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-parent</artifactId>
```

```
<version>3.4.0</version>
```

```
<relativePath/>
```

```
</parent>
```

Ereditiamo la configurazione di base da un «pom» genitore fornito direttamente da Spring Boot

```
...
```

```
<properties>
```

```
<java.version>17</java.version>
```

versione di Java del progetto

```
</properties>
```

```
<dependencies>
```

```
<dependency>
```

```
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-web</artifactId>
```

```
</dependency>
```

```
...
```

```
</dependencies>
```

Dipendenze del progetto: fanno riferimento a dei pacchetti «starter» predisposti da Spring Boot

Cenni al linguaggio XML – eXtensible Markup Language – I

XML è un **meta-linguaggio** di annotazione: non definisce quali tag posso usare (come invece fa HTML) ma permette di definire e usare i **propri tag**.

Perché un XML sia ben formato devono essere rispettati i seguenti vincoli:

- un tag XML deve essere una sequenza di caratteri alfanumerici fra parentesi angolari:
`< ... >`
- per ogni tag `<nome>` che scelgo di usare, il corrispondente tag di chiusura è `</nome>`
- un tag può **chiudersi** solo se tutti gli altri tag aperti al suo interno sono già stati chiusi; dunque l'XML (così come HTML) ha una **struttura ad albero**
- deve esserci **un unico tag radice** che contiene tutti gli altri

XML permette di **strutturare le informazioni** esprimendo il **significato** di ogni parte dell'informazione stessa

- già osservando il **pom.xml**: se eliminassimo i tag e la struttura sarebbe difficilissimo capire il senso delle sue varie parti!
- la strutturazione dell'XML rende le informazioni più facilmente **parsificabili** e **interpretabili** per **via automatica**
- al tempo stesso, grazie alla **maggiore leggibilità** per gli umani, riduce il rischio di errori

Cenni al linguaggio XML – eXtensible Markup Language – II

Quando si usa XML per uno scopo specifico, si può definire l'insieme dei tag ammissibili, e come devono/possono essere in relazione fra loro. Una definizione di questo tipo è detta **XML-Schema**

- un XML può fare riferimento a uno specifico XML-Schema nella sua intestazione
- in seguito a ciò potrà poi risultare **valido** o meno rispetto a quello schema
- un XML può essere **ben formato** come XML generico, ma **non valido** rispetto a un dato schema

Il file **pom.xml** fa riferimento allo schema

<https://maven.apache.org/xsd/maven-4.0.0.xsd>

Nota a margine: sarebbe in effetti quasi possibile definire HTML tramite un XML-Schema

- l'unico aspetto per cui un HTML **non** è un XML ben formato è che esistono tag **senza corrispondente chiusura**, come `<IMG>`
- l'HTML che si ottiene definendolo tramite XML (dunque, prevedendo la chiusura per qualsiasi tag) è detto **XHTML**
- la specifica XHTML ha per un po' convissuto, nel web, insieme a quella di HTML; è infine stata soppiantata da HTML Living Standard, nel quale sono validi i tag senza chiusura; ne consegue che un html **non** è un xml ben formato.

I «**dependency starter**» di **Spring Boot**

La gestione delle dipendenze è un tema rilevante nei progetti server-side basati su Java indipendentemente dal fatto che usino semplicemente il **Jakarta Web Profile** piuttosto che **Spring** o **Spring Boot** (così come d'altra parte lo è nei progetti client-side indipendentemente dal fatto che usino un framework o l'altro o nessuno).

Uno degli **asset** di **Spring Boot** sono i **dependency starter**, ossia dei «pacchetti di dipendenze» che corrispondono alle necessità più comuni delle applicazioni «enterprise» realizzate con Java, e che possono essere richiesti in blocco tramite il `pom.xml`, senza dover indicare uno per uno tutti i package necessari.

Per apprezzare questa caratteristica possiamo andare sul repository di **maven** (`mvnrepository.com`) e esaminare il loro contenuto.

- il **pom genitore** del **pom di progetto**:

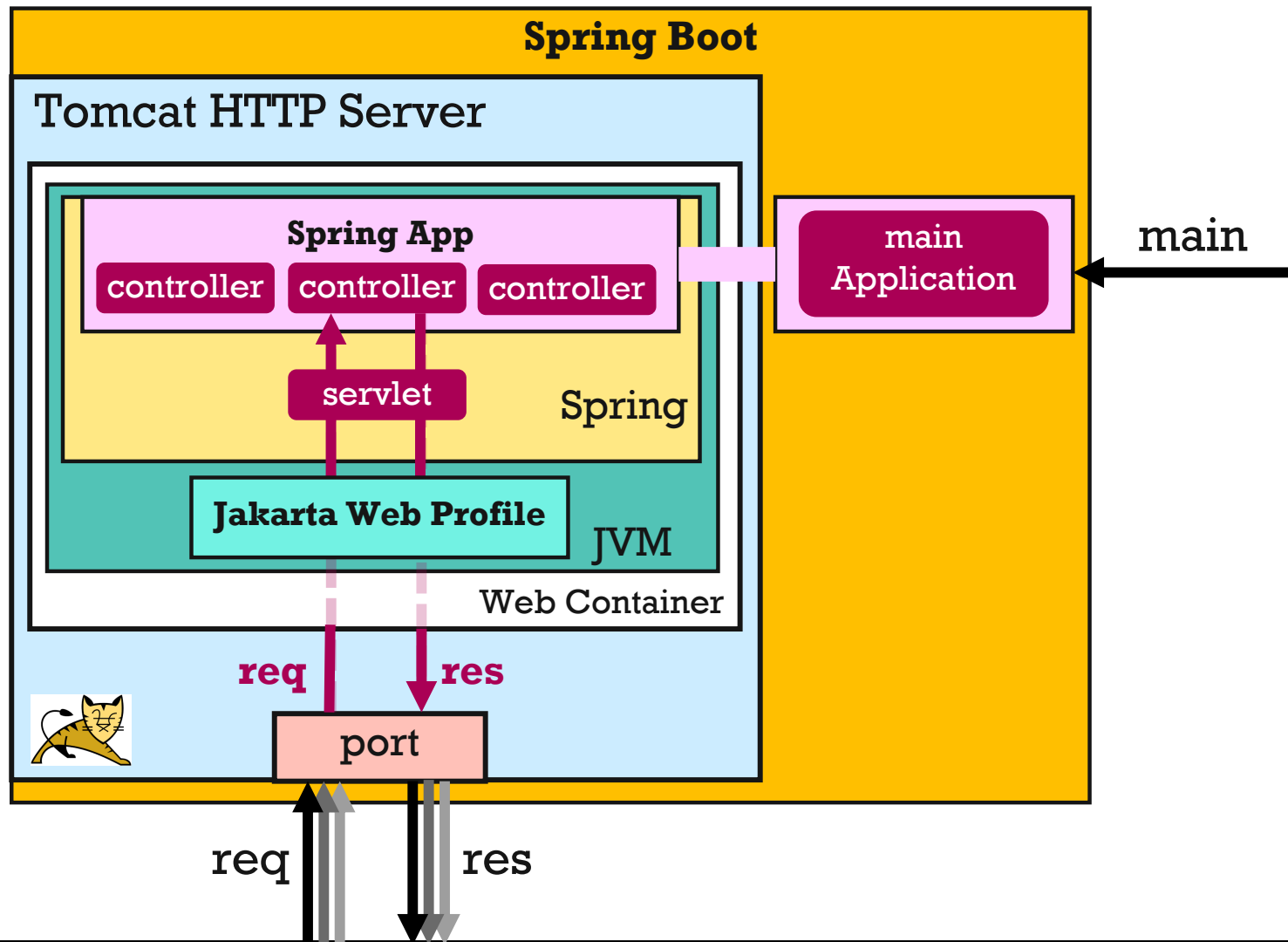
<https://mvnrepository.com/artifact/org.springframework.boot/spring-boot-starter-parent/3.4.0>

- il **dependency starter** per **web apps**

<https://mvnrepository.com/artifact/org.springframework.boot/web/3.4.0>

In entrambi i casi, scorrendo verso il basso, vediamo l'elenco delle singole dipendenze che compongono il pacchetto.

Spring-Boot: stand-alone app wrapping Tomcat + Spring



Applicazione server-side come app stand-alone

Se non usassimo Spring Boot, ma solo Spring, o solo il Jakarta Web Profile, la nostra applicazione non avrebbe una «**main class**» che fa da punto di ingresso dell'applicazione stessa. Sarebbe un insieme di classi che vengono istanziate dal Web Container (nel caso di una Jakarta EE) o dall'ambiente di esecuzione di Spring. Gli oggetti ottenuti dall'istanziamento verrebbero poi invocati quando necessario (ad esempio, quando c'è da gestire una richiesta HTTP).

Per eseguire l'applicazione sarebbe necessario

- installare Tomcat sul nostro computer
- compilare e «impacchettare» (**bundle**) il file .class così ottenuti
- effettuare il «rilascio» (**deploy**) del pacchetto in Tomcat, in modo che Tomcat possa eseguirlo all'interno del Web Container tramite la sua JVM

Tutto questo è estremamente dipendente dal sistema operativo, dalla configurazione e dalla versione in uso di Tomcat, ecc. ecc.

Spring Boot ci risparmia tutto questo.

Ci sono sempre le «classi operative» che vengono istanziate dall'ambiente di esecuzione di Spring, ed è sempre vero che le istanze ottenute vengono invocate quando necessario da quello stesso ambiente. Tuttavia c'è anche una **main class** che invoca le opportune librerie di Spring affinché eseguano una versione **embedded** di Tomcat con l'applicazione già rilasciata (**deployed**) al suo interno. Pertanto è possibile effettuare un semplice **run** dell'applicazione come faremmo con una qualsiasi applicazione Java.

Spring-Boot: stand-alone app

La main Application class

```
@SpringBootApplication
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```